Circuit Breaker Decorator

In microservices architecture, a circuit breaker is used to stop requests to a failing service to prevent cascading failures. Reliability teams must implement these patterns to protect internal resources from being overwhelmed by downstream latency or errors.

Implement a custom exception `CircuitOpenError` and a `@circuit_breaker` decorator with type hints. The decorator must track the failure count of a specific function and trip into an "OPEN" state once a threshold is reached, blocking all subsequent calls until the state is reset by a successful execution.

Example 1 Input:

```
CALL service_a 2 FAIL CALL service_a 2 FAIL CALL service_a 2 FAIL
```

Output:

```
FAIL: Service Error FAIL: Service Error ERROR: Circuit is OPEN - Call Blocked
```

Explanation: The failure threshold is set to 2. The first and second calls fail, reaching the threshold. The third call is blocked because the circuit is now OPEN, triggering the custom `CircuitOpenError` message.

Example 2 Input:

```
CALL service_b 2 FAIL CALL service_b 2 OK CALL service_b 2 FAIL
```

Output:

```
FAIL: Service Error Success FAIL: Service Error
```

Explanation: The first call fails. The second call succeeds, which resets the failure count for `service_b` back to zero. The third call is a new failure but does not trip the circuit yet because the threshold is 2.

Function Parameter:

- `failure_threshold` (int): The number of consecutive failures allowed before the circuit trips.

Returns

- The original function's return value on success.
- Raises `CircuitOpenError` when the circuit is in an OPEN state.
- Re-raises the original exception when a call fails in the CLOSED state.

Constraints

- $1 \le failure_threshold \le 10$
- The decorator must use `functools.wraps` to preserve function metadata.
- State must be independent for each decorated function.

Input Format for Custom Testing Input from stdin will be processed as follows: Each line contains a `CALL` command followed by `service_name`, `failure_threshold`, and the simulation action (`FAIL` or `OK`).

Sample Case 0 Sample Input

```
CALL service_c 1 FAIL CALL service_c 1 FAIL CALL service_d 1 OK
```

Sample Output

```
FAIL: Service Error ERROR: Circuit is OPEN - Call Blocked Success
```

Explanation Service C trips immediately after 1 failure. The second call to Service C is blocked. Service D is independent and succeeds normally.